

MACHINE LEARNING

EEG Signals analysis



POULET Lancelin
PASCHE Martín

Contents

1	Introduction	2
2	Data treatment	3
2.1	Features extraction	3
2.2	Features selection	5
2.3	Data augmentation	6
3	Method	6
3.1	Random Forest	6
3.2	Other approaches	7
3.3	Performance Metrics	7
3.4	Validation Protocol	7
4	Results	8
5	Conclusion	9

1 Introduction

During sleep the brain produces different types of signals. They represent the different periods of sleep : **Rapid Eye Movement** (RPE) and **Non Rapid Eye Movement** (NRE).

NREM sleep is divided into three stages:

- N1 (Drowsiness): A transitional stage between wakefulness and sleep, characterized by a reduction of alpha waves (8-13 Hz) and the appearance of theta waves (4-7 Hz).
- N2 (Light sleep): Characterized by sleep spindles (12-16 Hz) and K-complexes, which are transient high-amplitude peaks. This is the most commonly observed stage during sleep.
- N3 (Deep sleep): Also called slow-wave sleep (SWS), it is characterized by slow delta waves (0.5-4 Hz) of high amplitude. This stage is essential for physical recovery and memory consolidation.

REM sleep (Rapid Eye Movement sleep): During this phase, the brain is very active, producing EEG waves similar to those seen in wakefulness, often rapid mixed waves of low amplitude. The REM phase is associated with vivid dreaming and plays a key role in memory consolidation and cognitive processes.

There are various methods to study brain signals: **Electroencephalogram** (EEG), **Magnetoencephalography** (MEG), **functional Magnetic Mesonance Mmaging** (f-MRI), **function Near-Infrared Spectroscopy** (fNIRS) and **Positron Emission Tomography** (PET). But EEG are recognized as the strongest and most significant biomedical signal [1].

From now on, specialists were analyzing by hand these signals to understand the brain's behavior. The development of Machine Learning introduced the idea of an automatic labeling method. That is the objective of this project, using EEG signals, find a model to automatically classify a signal whether it is good or it contains artifacts.

The two types of signals to be predicted are :

- **Good quality** (class 1) : Sleep and wake-related EEG patterns are visible on the channel. They are not created by other noise sources (ECG/Respiration/... or non-physiological noise). Perturbation of these patterns over less than one second or by artifacts that appear irregularly at a rate less than 1hz is not considered as hindering interpretability.
- **Bad quality** (class 0) : Sleep patterns are not visible because of artifacts that make their reading and interpretation impossible, OR sleep-like patterns created by non-EEG/EOG sources and confuse the interpretation of the EEG.



Figure 1: Headband electrodes

The data are obtained with the Dream Handband from Beacon Biosignals, as shown in Figure 1. The signals have been captured with a sampling frequency of 250Hz, and the labeling is done on 2 seconds windows.

This project is based on the Kaggle competition : [EEG signal quality analysis](#).

2 Data treatment

The raw signals consists of 4 sets of 5 channels each, with a variable duration of few hours at 250Hz and labeled at 2 seconds windows. The 5 channels are the 5 electrodes of the Headband. To facilitate features extraction, we concatenated all channels following each other and each set also. Then all signals were sampled at 500 data points (2 seconds windows at 250Hz = 500 points). Resulting a single training dataset of 261755 lists containing 500 points each. The dataset for the test has been created with the same technique and is composed of 112615 samples of 60 columns also.

Before extracting all the important features, the signal has been filtered using a **Butter Band-pass Filter**. This filter is used to isolate a specific frequency range from a signal by filtering out frequencies outside the defined range. We used a lower cutoff frequency of 0.1Hz, and a Upper cutoff of 50Hz, with the 250Hz sampling frequency and choose a filter order of 4.

2.1 Features extraction

The dataset has been obtained by combining different approaches to extract features from the raw signals. In order to compute Machine Learning models we needed to create various indicators that will thus be our input variables.

From the work of Sen et al. [2] the variables can be separated in 4 groups : **Time domain features**, **Nonlinear based features**, **Frequency based features**, **Entropy-based features**. We obtained a total of 60 features : 13 in the time domain, 17 nonlinear, 22 in the frequency domain and 8 are entropy based features.

Category	Variable
Time Domain Features	Standard Deviation (std)
	Variance (var)
	Amplitude (amplitude)
	Skewness (skewness)
	Kurtosis (kurtosis)
	Minimum (min)
	Maximum (max)
	Mean (mean)
	Median (median)
	Amplitude Speed (amplitude_speed)
	Average Speed (avg_speed)
	Maximum Speed (max_speed)
	Minimum Speed (min_speed)
Nonlinear Based Features	Petrosian Fractal Dimension (pfd)
	Hjorth Complexity (hc)
	Hjorth Mobility (hm)
	Hjorth Activity (ha)
	Mean Curve Length (mcl)
	Zero Crossings (zc)
	Non-Negative Matrix Factorization (8 modes)
	Correlation with others channels
	Covariance with others channels
Frequency-Based Features	Euclidean Distance with others channels
	Power Spectral Density (delta, theta, alpha, beta)
	D3-1, D4-1, D5-1, A5-1 (Mean Absolute Value)
	D3-2, D4-2, D5-2, A5-2 (Average Power)
	D3-3, D4-3, D5-3, A5-3 (Standard Deviation)
	D3-4, D4-4 (Absolute Mean Ratios)
Entropy-Based Features	Wigner Ville (wv1 to wv4)
	Renyi Entropy (renyi_entropy)
	Spectral Entropy (spen)
	Permutation Entropy (pen)
	Approximate Entropy (apen)
	Mean Energy (me)
	Mean Teager Energy (mte)
	Hurst Exponent (hurst)
	Energy (energy)

Table 1: Features table updated with new variables

Each of the groups selected have specific reasons to be chosen and their utility for information extraction. This is illustrated on the following:

- **Time domain features** : These statistical measures capture the signal's overall behaviors over time. They are simple yet effective for distinguishing normal data from noise.

- **Nonlinear-based features** : These features reveal signal complexity and irregular patterns. They help detect subtle chaotic behaviors and long term dependencies.
- **Frequency-based features** : They analyze the signal's energy in specific frequency bands(e.g., delta, theta).
- **Entropy-based features** : They quantify the randomness and complexity of the signal. They measure the regularity and long-term relationships.

2.2 Features selection

The next step after extracting all these features from the signal is to select the most significant to our prediction. A popular and simple metric for this task is the **Fisher score test** [2]. It measures the effectiveness between two classes. We can use this metric to rank our features from the most important to the less one. The Fisher score is calculated as :

$$F_i = \frac{\sum_{a=1}^C n_a (\mu_{i,a} - \mu_i)^2}{\sum_{a=1}^C n_a \sigma_{i,a}^2} \quad (1)$$

where μ_i was the mean of the features; n_a was the number of samples in the a^{th} class; $\mu_{i,a}$ was the mean of the features in the a^{th} class; and $\sigma_{i,a}^2$ was the variance of the features in the a^{th} class.

Using this score we created subsets of features adding incrementally variables to it, starting to the one with the highest Fisher score, and finishing with the less. To find the best number of features we run a RandomForest with a 5-fold cross-validation to compute the Kappa score of each subset. But, as you can see in Figure 2, the Kappa score obtained for each subsets is increasing until all variables are used.

This is either due to a lack of significant variables or the overfitting of the model used to compute the Kappa score. Nonetheless, we decided to keep all these variables to create our model.

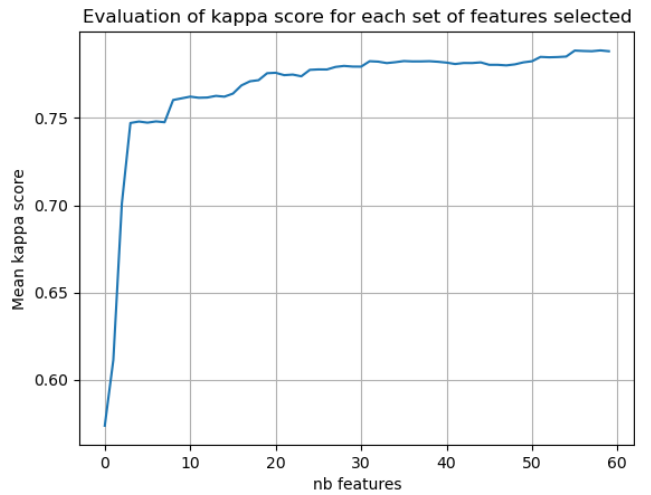


Figure 2: Kappa score through sets of features

2.3 Data augmentation

After extracting all the features, we obtained a dataset of 261755 samples and 60 columns. To lower the possible overfitting behavior of the model we choose to bootstrap it to increase the number of samples.

Bootstrap is a statistical resampling technique used to estimate the distribution of a statistic by repeatedly sampling with replacement from the original dataset. Each resampled dataset, called a "bootstrap sample," is the same size as the original.

Using this techniques we created 40000 other new samples. We choose 40000 to be less than the original dataset size, but to increase it so it can generalize more easily the model and have better performances on the testing set of 112615 samples. After bootstrap our training dataset is composed of 301755 samples of 60 columns. This induces a ratio of ~ 3 over the testing set.

3 Method

Many models were tested out to check which could achieve the highest score. In the end, the one which gave the best results, was random forest.

3.1 Random Forest

Before presenting our results, we will explain how Random Forest works. As the name suggests, this technique uses a collection of decision trees (the "forest") to make its final prediction. A decision tree is essentially a sequence of if-then-else conditions applied to the data. At each step (or split), the tree selects the feature that best separates the data into purer groups, meaning that after the split, each group contains the maximum possible proportion of similar outcomes (e.g., mostly "yes" or mostly "no"). This process reduces impurity, which is why we say that decision trees minimize impurity using metrics like Entropy or Gini Impurity.

Now, returning to Random Forest: this model builds many decision trees because it is a bagging technique, which combines bootstrap sampling and aggregation. In bootstrap sampling, multiple datasets (let's say M datasets) are created by randomly sampling with replacement from the original dataset. Each of these sampled datasets is used to train a separate decision tree. Once all the trees are trained, their predictions are combined using aggregation, typically through majority voting for classification tasks or averaging for regression tasks. A key detail is that, at each split, the decision trees in Random Forest do not consider all the features; instead, they choose from a random subset of features. This randomness introduces variability between the trees and helps prevent overfitting.

In summary, Random Forest is an effective technique for reducing variance and avoiding overfitting, while leveraging the power of a non-linear model to handle complex datasets. To construct our Random Forest model, we fixed the number of trees to 50, and

the maximal depth of each tree to 10. The number of trees depend of the complexity of the model, here 50 is effective enough. In, addition, fixing a limit of depth in the model is important to limit overfitting. Indeed, if the depth isn't fixed, the model may tend to learn too much details about the training dataset, thus lacking generalization.

3.2 Other approaches

Since Random Forest yielded promising results, the next logical step was to try XGBoost, a more advanced ensemble method. Given that this is a classification problem, we also tested logistic regression, a simple yet often effective baseline model, as sometimes the simplest solutions can perform best. Support Vector Machines (SVM) were then explored to identify clusters, but the algorithm proved too slow given the size of the dataset. Lastly, we experimented with Convolutional Neural Networks (CNNs), as they are well-suited for time series data. However, neural networks typically require large amounts of data to converge effectively, and this approach ultimately could not outperform Random Forest.

3.3 Performance Metrics

We evaluated model performance using Accuracy and Kappa (Cohen's Kappa), with the latter serving as the evaluation metric for the Kaggle test dataset. Other metrics such as Precision, Recall, or F1-score could also be considered but are not the focus here.

- **Accuracy** measures the proportion of correct predictions:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

where TP , TN , FP , and FN represent true positives, true negatives, false positives, and false negatives, respectively.

- **Kappa** quantifies agreement between two raters, adjusting for chance:

$$\kappa = \frac{P_o - P_e}{1 - P_e}$$

where P_o is the observed agreement:

$$P_o = \frac{TP + TN}{TP + TN + FP + FN}$$

and P_e is the expected agreement based on class proportions.

3.4 Validation Protocol

In the process of comparing models and fine-tuning the best ones, we implemented a cross-validation protocol. Cross-validation is a method used to assess a model's performance and

generalizability by repeatedly dividing the data into training and testing sets. This approach provides more reliable performance estimates by reducing bias and variance compared to a single train-test split.

For our model, we used a **5-fold cross-validation** to ensure robust evaluation.

4 Results

After training a Random Forest on our dataset, we were able to compute performance metrics. It is important to analyze these metrics for the testing and training set also. With this, we can understand if the model is overfitting or not.

Table 2 shows the metrics obtained for our Random Forest model. The accuracy for the test (0.89) is a little bit under what has been previously obtained with that type of method [2]. Training and testing accuracy are quite similar, but the training one is a little bit higher. This isn't problematic, but it could be a sign of overfitting.

The OOB score (Out-Of-Bag) is a performance metric used in Random Forest models to evaluate their accuracy without relying on a separate validation set. It provides a way to estimate the generalization error of the model during training. Here its value is the same as the test accuracy, which is also a good indicator about the generalization of the model.

Finally, the Kappa score gives good performance insight about the model. But when using the model with the Kaggle test dataset, the Kappa score decreases to : **Kappa = 0.73616**. This may be due to a lack of generalization of the model, even with the methods used to prevent overfitting.

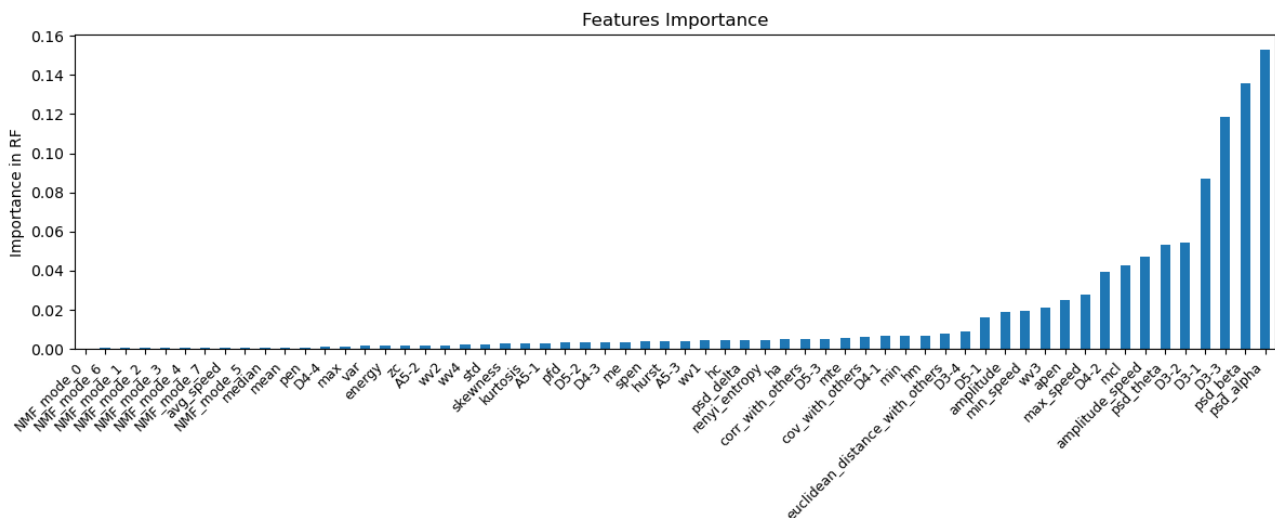
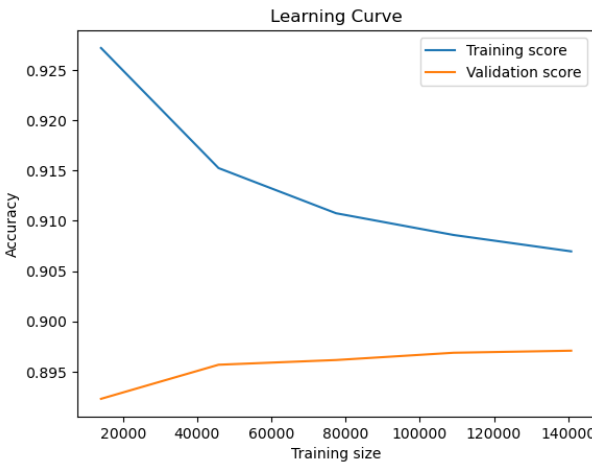
Table 3 shows the difference in the classification report for the training and testing dataset. Again, the performances seem to be similar which is a good indicator of generalization. An interesting point is that the accuracy on the class 1 is higher than with class 0. This shows that the model lacks of precision to detect a bad quality signal.

Table 2: Global Scores Summary

Metric	Value	Std
Train Accuracy	0.9061	± 0.027
Test Accuracy	0.8984	
OOB Score	0.8976	
Score Kappa	0.7889	± 0.058

Table 3: Classification Reports

Train Classification Report				Test Classification Report			
Class	Precision	Recall	F1-Score	Class	Precision	Recall	F1-Score
0	0.88	0.89	0.88	0	0.87	0.88	0.87
1	0.92	0.92	0.92	1	0.92	0.91	0.91
Accuracy	0.91 (211228 samples)			Accuracy	0.90 (90527 samples)		
Macro Avg	0.90	0.90	0.90	Macro Avg	0.89	0.89	0.89
Weighted Avg	0.91	0.91	0.91	Weighted Avg	0.90	0.90	0.90



Feature extraction proved to be a pivotal step in improving model accuracy. It involved identifying and providing the model with the most informative and relevant data, making it easier for the algorithm to detect patterns. To succeed in this step, we consulted related research papers to identify features that would enhance our task. While some patterns are visible in graphs, many remain hidden in high-dimensional spaces, reinforcing the importance of this process.

Next, we focused on finding the best model for our classification problem. Ultimately, we chose Random Forest, one of the most widely used models today, which delivered strong results using Cohen's Kappa as the evaluation metric. The model isn't performing as wanted, indeed it seems to overfit and lack of generalization. To improve it, we could find a better way to treat the features, and chose an Artificial Neural Network to capture more complex relations.

Although we did not achieve the best results in the course, the project inspired us to keep learning, researching, and exploring new methods to improve performance. Machine Learning is, after all, a continuously evolving field.

To manage all our researches during the project, and to work in team, we used GitHub. With that, we can monitor our progress and understand more easily the path of the project. This is the link to it : [GitHub Repository](#).

References

- [1] Wafaa S. Almuhammadi Khald Ali I. Aboalayon Miad Faezipour and Saeid Moslehpour. "Sleep Stage Classification Using EEG Signal Analysis: A Comprehensive Survey and New Investigation". In: *entropy* 18 (2016), p. 272.
- [2] B. Sen et al. "A Comparative Study on Classification of Sleep Stage Based on EEG Signals Using Feature Selection and Classification Algorithms". In: *J. Med. Syst.* 38 (2014), pp. 1–31.